

Documentación Técnica — Fase 2

Modos de Ejecución y Llamadas al Sistema (Syscalls)

Proyecto: OS bare-metal sobre BeagleBone Black (AM335x, ARM Cortex-A8) **Fase 2:** Frontera usuario/kernel, ABI de syscalls y contención de fallos **Objetivo principal:** BeagleBone (con build paralelo verificado en QEMU)

1. Resumen ejecutivo

La Fase 1 demostró que la **multiprogramación preemptiva funciona**: un *timer* (DMTimer2) genera interrupciones periódicas y un planificador *Round-Robin* alterna entre procesos haciendo *context switch*.

La Fase 2 eleva eso a **"la conmutación preemptiva funciona bajo control protegido del kernel"**. Se agregan tres cosas:

- Frontera usuario/kernel estricta.** Los procesos de usuario (P1, P2, P3) ahora corren en modo **USR** (no privilegiado). El kernel corre en modos privilegiados.
- ABI mínima de syscalls.** El usuario solo pide servicios al kernel mediante la instrucción `svc #0` con una convención de registros (`yield`, `exit`, `write`).
- Aislamiento por fallos.** Si una tarea de usuario ejecuta una instrucción ilegal o accede a memoria inválida, el kernel la **clasifica, termina y aísla**, sin que se caiga el sistema; las tareas sanas continúan.

Todo esto preservando el núcleo de la Fase 1 (timer + round-robin + context switch).

2. Arquitectura y estructura de directorios

El proyecto mantiene una separación por capas (HAL / OS / librería / usuario):

```
OS-PJ2-CC/
├── Makefile                # Compila OS + P1/P2/P3 (BeagleBone por defecto, QEMU opcional)
├── hal/                    # Hardware Abstraction Layer (depende de la placa)
│   ├── beagle/
│   │   ├── root.s          # *** Vector table + reset + handlers (IRQ/SVC/abort) ***
│   │   ├── uart.c          # UART0 (0x44E09000)
│   │   ├── timer.c         # DMTimer2 (0x48040000) + INTC (0x48200000)
│   │   ├── watchdog.c      # WDT1 (deshabilitar para que no reinicie)
│   │   └── qemu/           # Equivalentes para QEMU VersatilePB (ARM926)
│   │       ├── root.s
│   │       └── ...
├── os/                    # Núcleo independiente de la placa
│   ├── os.c                # main(), timer_irq_handler(), os_idle()
│   ├── scheduler.c/.h      # init_scheduler(), schedule(), terminate_process()
│   ├── syscall.c           # svc_handler() (dispatcher de syscalls) [NUEVO Fase 2]
│   ├── fault.c             # fault_handler() (contención de fallos) [NUEVO Fase 2]
│   ├── context.h           # frame_to_pcb()/pcb_to_frame() compartidos [NUEVO Fase 2]
│   ├── pcb.h               # struct PCB + estados + códigos de fallo
│   └── os.ld               # Linker script del OS (direcciones y stacks)
├── lib/
│   ├── stdio.c/.h          # PRINT (usado SOLO por el kernel)
│   └── user_syscalls.h      # Wrappers de syscalls (usado SOLO por el usuario) [NUEVO]
├── P1/ (main.c, p1.ld)     # Proceso 1: imprime dígitos 0-9 (write + yield, infinito)
├── P2/ (main.c, p2.ld)     # Proceso 2: letras a-z, pruebas de error, luego exit
└── P3/ (main.c, p3.ld)     # Proceso 3: corre y luego provoca un fallo [NUEVO Fase 2]
```

Idea clave de diseño: el código de usuario **no enlaza nada del kernel**. P1/P2/P3 solo incluyen `lib/user_syscalls.h` (puro `inline asm`), y en el `Makefile` cada binario de usuario enlaza únicamente su propio `main.o`. Esto hace literal la frontera: el usuario *no tiene forma* de tocar hardware salvo a través de `svc`.

3. Mapa de memoria (sin MMU, direcciones fijas)

Como no hay MMU, todo se enlaza a direcciones fijas que no se solapan.

BeagleBone (DRAM en 0x80000000)

Región	Inicio	Tamaño	Descripción
OS código/datos	0x82000000	64 KB	.text/.data/.bss del kernel
Stack IRQ	tope 0x82010000	—	Modo IRQ (bancado)
Stack SVC (OS)	tope 0x82012000	8 KB	El OS arranca aquí en main()
Stack System/idle	tope 0x82014000	8 KB	Comparte SP/LR con USR; lo usa os_idle()
Stack Abort	tope 0x82015000	4 KB	Handlers de prefetch/data abort
Stack Undefined	tope 0x82016000	4 KB	Handler de instrucción indefinida
P1 código/datos	0x82100000	64 KB	Proceso 1
P1 stack	tope 0x82112000	8 KB	
P2 código/datos	0x82200000	64 KB	Proceso 2
P2 stack	tope 0x82212000	8 KB	
P3 código/datos	0x82300000	64 KB	Proceso 3
P3 stack	tope 0x82312000	8 KB	

Región válida de usuario (para validar punteros en SYS_WRITE): [0x82100000, 0x82320000).

QEMU VersatilePB (RAM en 0x0)

Equivalente con bases 0x00100000 (P1), 0x00200000 (P2), 0x00300000 (P3); stacks +0x2000. Región de usuario [0x00100000, 0x00320000).

4. Modos de ejecución ARM usados

ARMv7-A tiene varios modos de procesador. Cada modo (salvo USR/SYS que comparten) tiene su propio SP y LR bancados (registros físicos distintos).

Modo	CPSR[4:0]	Privilegio	Para qué lo usamos
USR	0x10	No privilegiado	P1, P2, P3 (código de usuario)
FIQ	0x11	Privilegiado	(no usado)
IRQ	0x12	Privilegiado	Handler del timer (irq_handler)
SVC	0x13	Privilegiado	Arranque del OS y handler de svc
ABT	0x17	Privilegiado	Handlers de prefetch/data abort
UND	0x1B	Privilegiado	Handler de instrucción indefinida
SYS	0x1F	Privilegiado	os_idle y rescate del SP/LR de USR

Bits relevantes del CPSR: I (bit 7, 0x80) enmascara IRQ; F (bit 6, 0x40) enmascara FIQ; T (bit 5) = estado Thumb.

Punto fino central de la Fase 2: USR y System **comparten** el mismo SP/LR bancado. Por eso, para guardar/restaurar el SP y LR de una tarea de usuario desde un handler privilegiado, cambiamos temporalmente a **System mode (0x1F)**, no a SVC. El valor que cargamos a cpsr_c es 0x9F = 0x1F (System) | 0x80 (I=1, IRQ deshabilitada durante la sección crítica).

5. El "frame" de 17 palabras (contrato compartido)

Las cuatro rutas de entrada al kernel (arranque, IRQ, syscall, abort) construyen **exactamente la misma** estructura de 17 palabras en su stack de modo, y pasan un puntero a ella a la función C. Así, una sola lógica de guardar/restaurar (context.h) sirve para todas.

```

índice  contenido
frame[0] = SP de la tarea  (bancado USR/System)
frame[1] = LR de la tarea
frame[2] = SPSR             (CPSR de la tarea: lleva los bits de modo USR)
frame[3] = r0
frame[4] = r1
...
frame[15] = r12
frame[16] = PC              (dirección de reanudación / instrucción que falló)

```

Construcción típica en ensamblador (orden de los `stmfd`):

```

stmfd sp!, {r0-r12, lr}  @ guarda r0..r12 + PC corregido  (14 palabras)
mrs  r0, spsr
stmfd sp!, {r0}          @ guarda SPSR                      (15)
@ ... cambia a System mode, lee SP/LR de la tarea en r2,r3 ...
stmfd sp!, {r2, r3}      @ guarda SP y LR                  (17) -> frame completo
mov  r0, sp              @ r0 = &frame (argumento para C)

```

El PCB (`pcb.h`) refleja este mismo orden, por lo que copiar `frame -> PCB` y `PCB -> frame` es trivial (ver `context.h`).

6. Las rutas de cruce y el catálogo `MODE_SWITCH`

Cada cruce USR↔kernel emite una línea de traza por UART, identificable en los logs. Catálogo completo (Sección 3.8 del enunciado):

#	Ruta	Dirección	Cuándo	Traza
1	Arranque	Kernel→User	Primer salto a USR	<code>MODE_SWITCH KERNEL_TO_USER pid=<f> reason=initial_launch</code>
2	Interrupción	User→Kernel	IRQ del timer en USR	<code>MODE_SWITCH USER_TO_KERNEL pid=<n> reason=timer_irq</code>
3	Interrupción	Kernel→User	Tras planificar	<code>MODE_SWITCH KERNEL_TO_USER pid=<m> reason=dispatch</code>
4	Syscall	User→Kernel	<code>svc</code> desde USR	<code>MODE_SWITCH USER_TO_KERNEL pid=<n> reason=syscall id=<id></code>
5	Syscall	Kernel→User	Tras servir el syscall	<code>MODE_SWITCH KERNEL_TO_USER pid=<m> reason=syscall_return id=<id> rc=<rc></code>
6	Excepción	User→Kernel	Abort/undef desde USR	<code>MODE_SWITCH USER_TO_KERNEL pid=<n> reason=fault type=<tipo></code>
7	Excepción	Kernel→User	Tras política de fallo	<code>MODE_SWITCH KERNEL_TO_USER pid=<m> reason=fault_recovery</code>

Offsets de LR (críticos) — cuánto restar a `lr` para apuntar a la instrucción correcta al volver:

Vector	Modo	Offset	Razón
IRQ (0x18)	IRQ	<code>-4</code>	<code>lr_irq</code> = instrucción interrumpida + 4
SVC (0x08)	SVC	<code>0</code>	<code>lr_svc</code> ya apunta a la instrucción <i>siguiente</i> a <code>svc</code>
Prefetch abort (0x0C)	ABT	<code>-4</code>	<code>lr_abt</code> = instrucción que falló + 4
Data abort (0x10)	ABT	<code>-8</code>	<code>lr_abt</code> = instrucción que falló + 8
Undefined (0x04)	UND	<code>-4</code>	<code>lr_und</code> = instrucción indefinida + 4

7. ABI de Syscalls (entregable §7)

7.1 Convención de registros (ARM / SVC)

Registro	Entrada	Salida
r0	ID de syscall	valor de retorno (int32_t)
r1	argumento 1	—
r2	argumento 2	—
r3	argumento 3	—
instrucción	svc #0 (trap canónico, "SVCall")	

7.2 Tabla de identificadores

Símbolo	ID	Propósito
SYS_YIELD	0	Ceder la CPU voluntariamente (reprograma)
SYS_EXIT	1	Terminar al llamador; no retorna
SYS_WRITE	2	Escribir bytes a UART (fd debe ser 1)

7.3 Tabla de valores de retorno / errores

Valor	Significado
>= 0	Éxito (cantidad de bytes en write)
-1	ID de syscall inválido
-2	Descriptor (fd) o argumento inválido
-3	Puntero de usuario inválido o violación de protección

7.4 Comportamiento de cada syscall

- SYS_YIELD** : valida que vino de USR; marca al llamador **READY** ; corre el planificador (round-robin); restaura y vuelve a USR para la tarea elegida (puede no ser el llamador). Retorna **0** al llamador cuando este vuelve a correr.
- SYS_EXIT** : registra el código de salida; marca la tarea **TERMINATED** (se saca del conjunto ejecutable); planifica otra tarea; si no queda ninguna, entra a **os_idle** (política documentada de idle/halt). No retorna.
- SYS_WRITE** : valida **fd == 1** ; limita **len** a un máximo (4096); valida que **[buf, buf+len)** esté dentro de la región de usuario **antes** de desreferenciar; copia byte a byte a UART; retorna la cantidad escrita o un código de error.

7.5 ID desconocido / no-USR

Cualquier ID fuera de rango retorna **-1** sin alterar el kernel. Si el **svc** no proviene de USR (verificado leyendo el SPSR guardado, **(spsr & 0x1F) == 0x10**), se rechaza con **-1** . No existe ruta de escalada de privilegios.

8. Política de manejo de fallos (entregable §7)

Cuando una tarea de usuario causa una excepción síncrona, el CPU entra al vector correspondiente; el handler arma el frame y llama a **fault_handler(&frame, kind)** . Política: **clasificar** → **registrar** → **terminar** → **reprogramar** .

8.1 Matriz clasificación → resultado

Vector	kind	Registro de estado leído	Clasificación (type=)	Resultado
Undefined (0x04)	FAULT_UNDEF (6)	— (no aplica)	undef	Tarea TERMINATED , se reprograma un par sano
Prefetch abort (0x0C)	FAULT_PREFETCH (1)	IFSR (c5,c0,1), IFAR (c6,c0,2)	según FS	idem
Data abort (0x10)	FAULT_DATA (2)	DFSR (c5,c0,0), DFAR (c6,c0,0)	según FS	idem

8.2 Decodificación del campo FS (Fault Status, short-descriptor)

```
FS = FSR[3:0] | (FSR[10] << 4) :
```

FS	type=
0x01	alignment
0x05 , 0x07	translation
0x08 , 0x16	external
0x0D , 0x0F	permission
otro	other

8.3 Observabilidad

Además de las trazas 6 y 7, se emite una línea de diagnóstico:

```
FAULT pid=<n> kind=<k> pc=<pc> cpsr=<cpsr> status=<fsr> addr=<far>
```

El PCB de la tarea guarda `fault_type` y `exit_code` para inspección posterior.

9. PCB unificado (Sección 6)

`pcb.h` define una sola estructura que soporta las Secciones 3-5:

```
typedef struct {
    uint32_t pid;           // 0=OS/idle, 1=P1, 2=P2, 3=P3
    uint32_t state;         // READY / RUNNING / WAITING / TERMINATED
    uint32_t sp, lr, pc;    // contexto guardado (banco USR/System)
    uint32_t cpsr;          // CPSR de la tarea (modo/estado)
    uint32_t r[13];         // r0..r12
    int32_t exit_code;       // registrado por SYS_EXIT
    uint32_t fault_type;     // FAULT_* (razón de terminación)
    uint32_t last_syscall;   // último ID de syscall
} pcb_t;
```

Estados: `READY (0)`, `RUNNING (1)`, `WAITING (2)`, `TERMINATED (3)`. Códigos de fallo: `FAULT_NONE(0)`, `PREFETCH(1)`, `DATA(2)`, `ALIGN(3)`, `PRIVILEGE(4)`, `EXIT(5)`, `UNDEF(6)`.

10. Compilación y carga

Build

```
make           # BeagleBone (Cortex-A8) por defecto -> build/os.bin, p1.bin, p2.bin, p3.bin
make TARGET=QEMU # QEMU VersatilePB (ARM926)
make clean
```

El OS se enlaza con `os/os.ld` a su base; cada proceso con su propio `pX.ld` a su base.

Carga en placa (U-Boot, ejemplo)

```
loady 0x82000000 # enviar os.bin
loady 0x82100000 # enviar p1.bin
loady 0x82200000 # enviar p2.bin
loady 0x82300000 # enviar p3.bin
go 0x82000000 # arranca el OS; el primer tick del timer salta a P1 en USR
```

Ejecución en QEMU

```
qemu-system-arm -M versatilepb -nographic -kernel build/os.elf \  
-device loader,file=build/p1.bin,addr=0x00100000 \  
-device loader,file=build/p2.bin,addr=0x00200000 \  
-device loader,file=build/p3.bin,addr=0x00300000
```

11. Verificación (corrida en QEMU)

Secuencia observada (resumida):

```
MODE_SWITCH KERNEL_TO_USER pid=1 reason=initial_launch    <- ruta 1  
MODE_SWITCH USER_TO_KERNEL pid=1 reason=syscall id=2      <- write (ruta 4)  
----From P1: 0  
MODE_SWITCH KERNEL_TO_USER pid=1 reason=syscall_return id=2 rc=15 <- ruta 5  
...  
MODE_SWITCH USER_TO_KERNEL pid=2 reason=syscall id=2 ... rc=-2 <- fd inválido  
MODE_SWITCH KERNEL_TO_USER pid=2 reason=syscall_return id=2 rc=-3 <- puntero inválido  
...  
MODE_SWITCH USER_TO_KERNEL pid=2 reason=syscall id=1  
PROCESS_EXIT pid=2 code=0                                <- P2 termina (exit)  
...  
MODE_SWITCH USER_TO_KERNEL pid=3 reason=fault type=undef  <- ruta 6  
FAULT pid=3 kind=6 pc=3000a8 cpsr=20000010 status=0 addr=3000a8  
MODE_SWITCH KERNEL_TO_USER pid=1 reason=fault_recovery    <- ruta 7
```

Resultados clave:

- Las tres tareas corren en **modo USR** (lo confirma `cpsr=...0010` en la traza de fallo).
- `write` retorna la cuenta de bytes (`rc=15`); entradas inválidas retornan `rc=-2` y `rc=-3` **sin corromper el kernel**.
- `exit` saca a P2 permanentemente: **0** actividad de P2 después de `PROCESS_EXIT`.
- El fallo de P3 se **contiene**: P3 nunca vuelve a ejecutarse; el kernel procesó >124,000 eventos más y P1 siguió corriendo (>30,000 escrituras).
- Ambos targets compilan con `-Wall` y **cero advertencias**.